

Analyzing Spotify Music Genres Using Python Nathan Cao 6/26/2026

Introduction

The goal of this project is to compare Spotify music genres according to their audio characteristics. By analyzing variables such as popularity, danceability, energy, valence, tempo, loudness, and song duration, I will identify the characteristics that distinguish different genres and determine how genres differ from one another.

Here I am importing and loading the database.

```
In [38]: import pandas as pd
import matplotlib.pyplot as plt

spotify = pd.read_csv(r"C:\Users\natha\OneDrive\Documents\dataset.csv")

spotify.head()
```

Out[38]:

	Unnamed: 0	track_id	artists	album_name	track_name	popul
0	0	5SuOikwiRyPMVolQDJUgSV	Gen Hoshino	Comedy	Comedy	
1	1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	
2	2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN	To Begin Again	To Begin Again	
3	3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Sou...	Can't Help Falling In Love	
4	4	5vjLSffimilP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	

5 rows × 21 columns

Here I am examining the structure of the database to understand the variables, data types, and summary statistics.

```
In [39]: spotify.info()

spotify.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 114000 entries, 0 to 113999
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            114000 non-null int64
1   track_id              114000 non-null object
2   artists               113999 non-null object
3   album_name            113999 non-null object
4   track_name            113999 non-null object
5   popularity            114000 non-null int64
6   duration_ms           114000 non-null int64
7   explicit              114000 non-null bool
8   danceability          114000 non-null float64
9   energy                114000 non-null float64
10  key                   114000 non-null int64
11  loudness              114000 non-null float64
12  mode                  114000 non-null int64
13  speechiness           114000 non-null float64
14  acousticness          114000 non-null float64
15  instrumentalness       114000 non-null float64
16  liveness              114000 non-null float64
17  valence               114000 non-null float64
18  tempo                 114000 non-null float64
19  time_signature         114000 non-null int64
20  track_genre            114000 non-null object
dtypes: bool(1), float64(9), int64(6), object(5)
memory usage: 17.5+ MB
```

Out[39]:

	Unnamed: 0	popularity	duration_ms	danceability	energy	
count	114000.000000	114000.000000	1.140000e+05	114000.000000	114000.000000	114000.0
mean	56999.500000	33.238535	2.280292e+05	0.566800	0.641383	5.3
std	32909.109681	22.305078	1.072977e+05	0.173542	0.251529	3.5
min	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.0
25%	28499.750000	17.000000	1.740660e+05	0.456000	0.472000	2.0
50%	56999.500000	35.000000	2.129060e+05	0.580000	0.685000	5.0
75%	85499.250000	50.000000	2.615060e+05	0.695000	0.854000	8.0
max	113999.000000	100.000000	5.237295e+06	0.985000	1.000000	11.0

Here I am cleaning the data, preparing it for analysis. I wanted to check for missing values to see if it would affect the analysis. Then I wanted to remove any duplicate data to prevent them from influencing the results. Then I created a new variable to convert the songs from milliseconds to minutes because that is typically how we view the length of songs.

```
In [40]: spotify.isnull().sum()

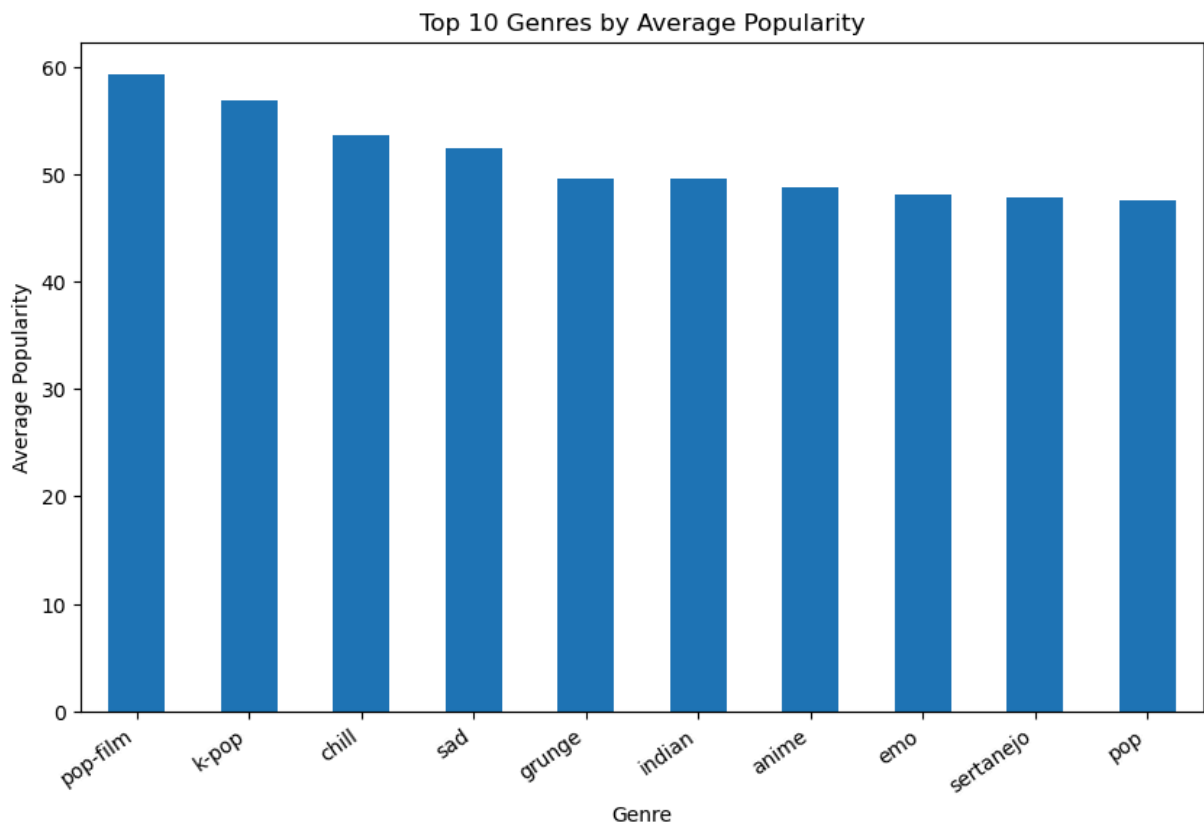
spotify.duplicated().sum()
```

```
spotify = spotify.drop_duplicates()

spotify["duration_min"] = spotify["duration_ms"] / 60000
```

```
In [41]: genrePopularity = spotify.groupby("track_genre")["popularity"] \
        .mean() \
        .sort_values(ascending=False) \
        .head(10)

plt.figure(figsize=(10,6))
genrePopularity.plot(kind="bar")
plt.title("Top 10 Genres by Average Popularity")
plt.xlabel("Genre")
plt.ylabel("Average Popularity")
plt.xticks(rotation=35, ha="right")
plt.show()
```

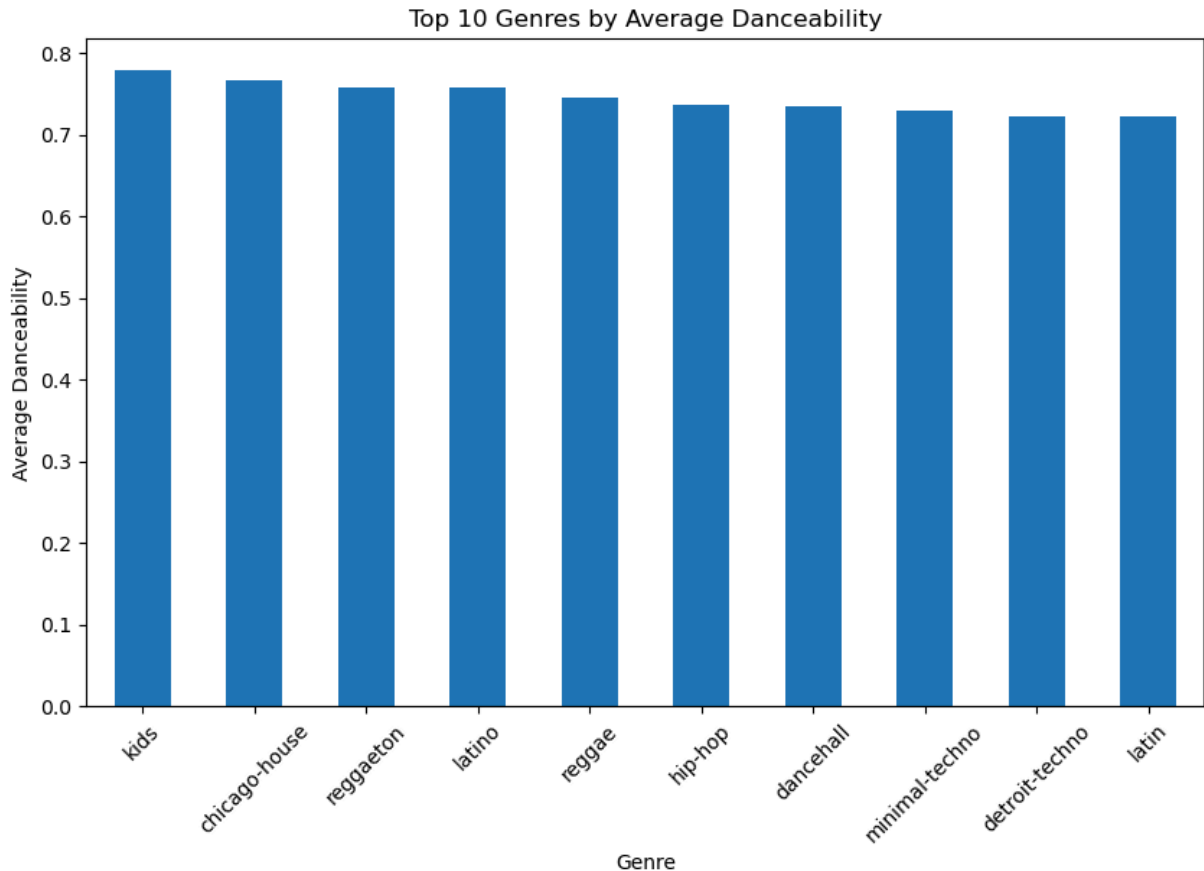


Here is a graph showing the top 10 genres by average popularity. As we can see, pop film was the most popular.

```
In [42]: genreDance = spotify.groupby("track_genre")["danceability"].mean().sort_values(ascending=False)

plt.figure(figsize=(10, 6))
genreDance.plot(kind="bar")
plt.title("Top 10 Genres by Average Danceability")
plt.xlabel("Genre")
plt.ylabel("Average Danceability")
```

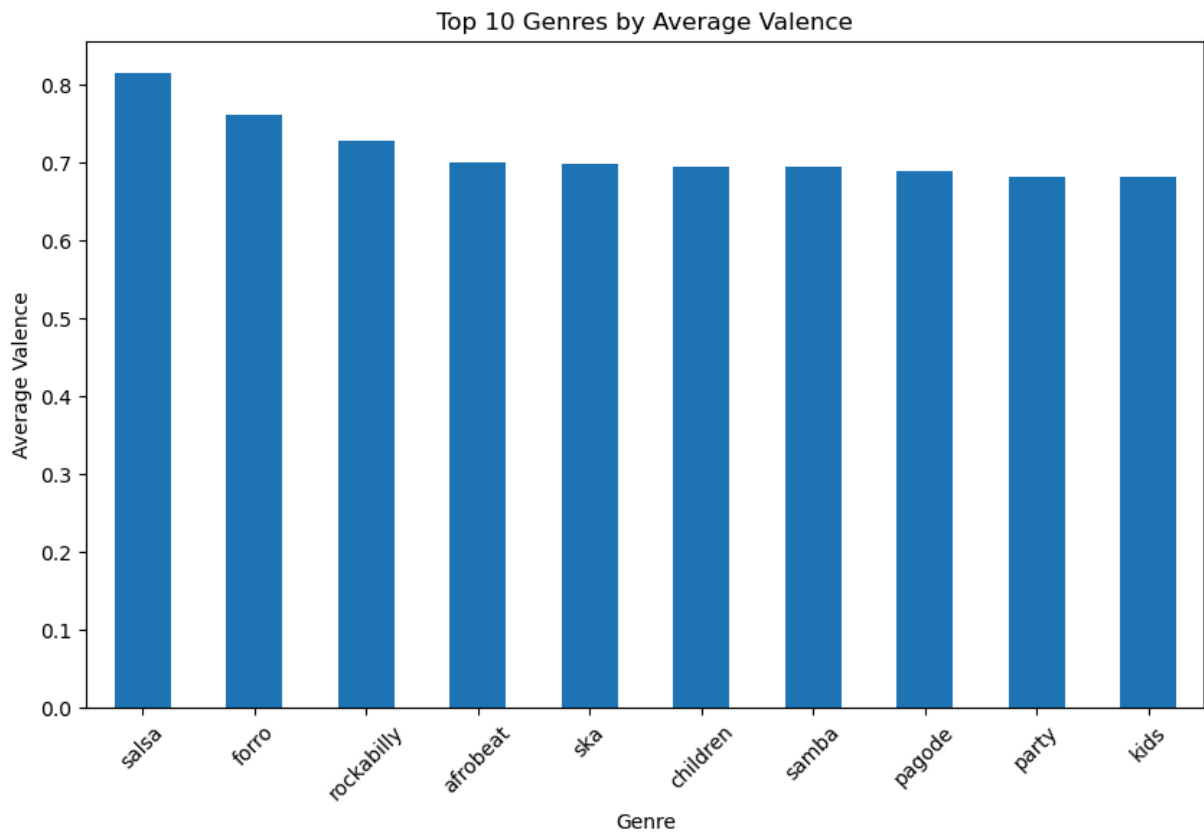
```
plt.xticks(rotation=45)
plt.show()
```



For genres that are the most danceable. We can see that kids scored the highest. The top 10 genres are relatively close. But what they have in common is that they have steady beats. A lot of the genres in the top 10 are closely related or sub-genres of EDM as well.

```
In [43]: genreValence = spotify.groupby("track_genre")["valence"].mean().sort_values(ascendi

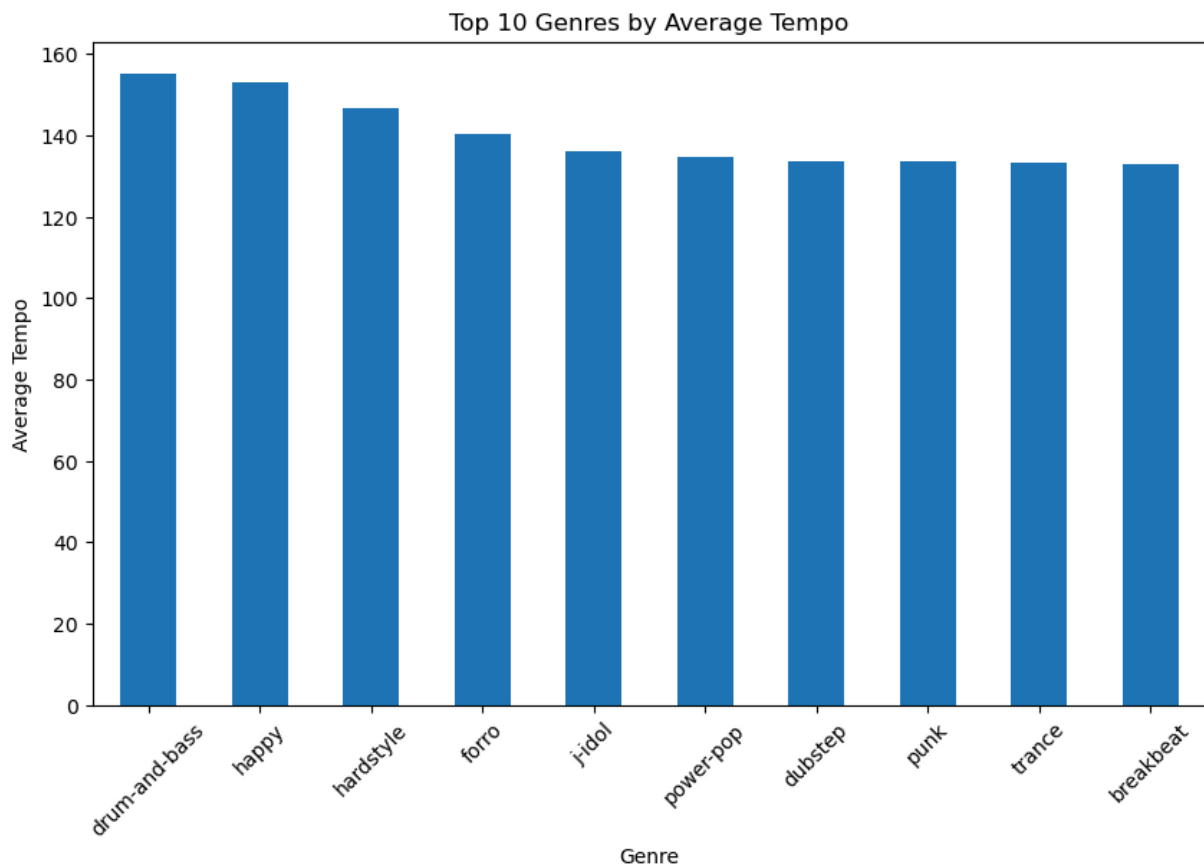
plt.figure(figsize=(10, 6))
genreValence.plot(kind="bar")
plt.title("Top 10 Genres by Average Valence")
plt.xlabel("Genre")
plt.ylabel("Average Valence")
plt.xticks(rotation=45)
plt.show()
```



This graph shows the average valence or brightness in the music genres. Salsa has the highest average likely due to its happiness and energeticness.

```
In [44]: genreTempo = spotify.groupby("track_genre")["tempo"].mean().sort_values(ascending=False)

plt.figure(figsize=(10, 6))
genreTempo.plot(kind="bar")
plt.title("Top 10 Genres by Average Tempo")
plt.xlabel("Genre")
plt.ylabel("Average Tempo")
plt.xticks(rotation=45)
plt.show()
```



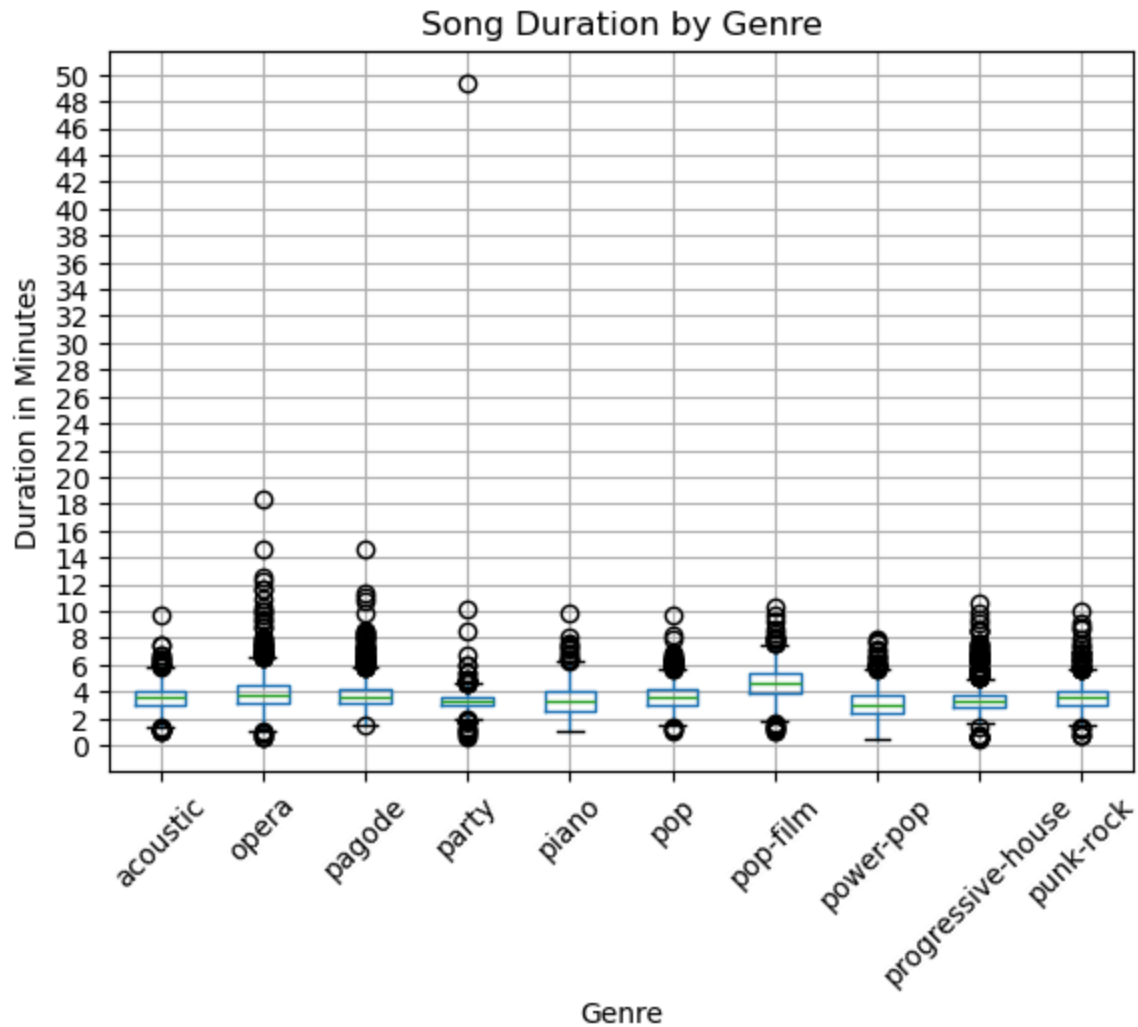
This graph shows that drum-and-bass has the highest tempo. Closely following that genre is happy and hardstyle.

```
In [45]: spotifyTopGenres.boxplot(column="duration_min", by="track_genre")

plt.title("Song Duration by Genre")
plt.suptitle("")
plt.xlabel("Genre")
plt.ylabel("Duration in Minutes")

plt.xticks(rotation=45)
plt.yticks(range(0, 52, 2)) # Change increments here

plt.show()
```



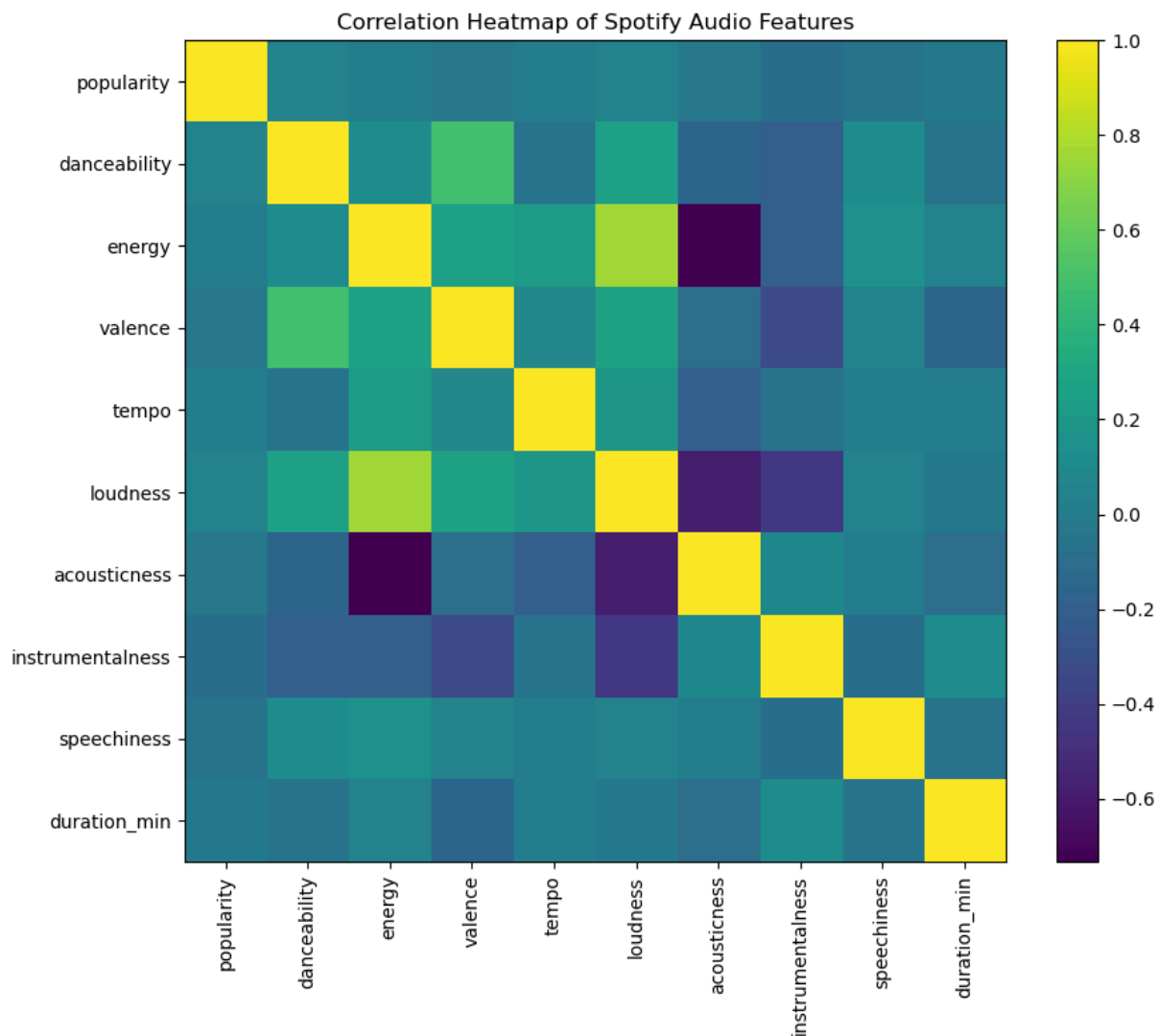
This boxplot graph compares the genres by duration. The median duration is between 3 and 4 minutes. This means that the duration of the music is consistent regardless of the genre.

```
In [46]: audioFeatures = [
    "popularity",
    "danceability",
    "energy",
    "valence",
    "tempo",
    "loudness",
    "acousticness",
    "instrumentalness",
    "speechiness",
    "duration_min"
]

corr = spotify[audioFeatures].corr()

plt.figure(figsize=(10, 8))
plt.imshow(corr)
plt.colorbar()
plt.xticks(range(len(audioFeatures)), audioFeatures, rotation=90)
plt.yticks(range(len(audioFeatures)), audioFeatures)
```

```
plt.title("Correlation Heatmap of Spotify Audio Features")
plt.show()
```



This heatmap shows the correlation between different spotify audio features. The closer to 1 indicates a strong positive relation and viseversa for -1. If the value is close to 0, there is no correlation.

```
In [47]: genreProfile = spotify.groupby("track_genre")[
    ["popularity", "danceability", "energy", "valence", "tempo", "duration_min"]
].mean().round(2)

genreProfile.head(10)
```


Out[47]:

	popularity	danceability	energy	valence	tempo	duration_min
track_genre						
acoustic	42.48	0.55	0.44	0.42	119.01	3.58
afrobeat	24.40	0.67	0.70	0.70	119.21	4.14
alt-rock	33.94	0.53	0.75	0.52	124.63	3.92
alternative	24.34	0.56	0.72	0.50	122.23	3.70
ambient	44.19	0.37	0.24	0.17	111.11	3.95
anime	48.77	0.54	0.67	0.43	123.53	3.50
black-metal	22.45	0.30	0.87	0.19	128.60	5.19
bluegrass	25.68	0.53	0.53	0.64	126.58	3.69
blues	31.19	0.57	0.58	0.60	116.57	3.71
brazil	44.67	0.56	0.62	0.47	121.94	4.57

The table above shows the average audio characteristics for each Spotify music genre. Each row represents a genre, and each column displays the average value of a particular audio feature.

Spotify music genres were analyzed using several audio features of the song, including popularity, danceability, energy, valence, tempo, and duration. The analysis revealed significant differences in musical characteristics across genres. Some genres are more upbeat and have a faster tempo, whereas others are more relaxed, acoustic, or danceable. Additionally, the correlation analysis showed that some features, such as energy and loudness, are strongly correlated, while popularity is weakly correlated with the majority of audio features. Finally, the project demonstrates how to use Python data analysis to identify musical patterns and compare genres based on measurable audio characteristics.

Sorce: <http://kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>

This is the database I used for this project.